# Brief Announcement: The MBroadcast Abstraction

Mathilde Déprés
ENS Paris-Saclay, Université Paris-Saclay
91190 Gif-sur-Yvette, France
mathilde.depres@ens-paris-saclay.fr

Achour Mostéfaoui
LS2N, Nantes Université
44322 Nantes, France
achour.mostefaoui@univ-nantes.fr

Matthieu Perrin
LS2N, Nantes Université
44322 Nantes, France
matthieu.perrin@univ-nantes.fr

Michel Raynal
Univ Rennes IRISA, Inria, CNRS
Rennes, 35042 France, France
michel.raynal@irisa.fr

## ABSTRACT

This short article presents a new communication abstraction denoted *Mutual Broadcast* (in short MBroadcast). It provides each pair of processes with the following property (called *mutual ordering*): for any pair of processes $p$ and $p'$, if $p$ broadcasts a message $m$ and $p'$ broadcasts a message $m'$, it is not possible for $p$ to deliver first (its message) $m$ and then $m'$ while $p'$ delivers first (its message) $m'$ and then $m$. The computability power of this broadcast abstraction is the same as the one of an atomic read/write register. Interestingly, it constitutes the first characterization of RW registers in terms of (binary) message patterns.

## CCS CONCEPTS

• **Theory of computation → Distributed computing models**; • **Computer systems organization → Fault-tolerant network topologies**; • **Networks** → *Programming interfaces.*

## KEYWORDS

Asynchrony, Atomicity, Broadcast abstraction, Characterization, Crash failure, Distributed Computability, Message-passing, Message pattern, Quorum, Read/write pattern, Read/Write register, Two-process communication.

**ACM Reference Format:**
Mathilde Déprés, Achour Mostéfaoui, Matthieu Perrin, and Michel Raynal. 2023. Brief Announcement: The MBroadcast Abstraction. In *ACM Symposium on Principles of Distributed Computing (PODC '23), June 19–23, 2023, Orlando, FL, USA.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3583668.3594569

## 1 INTRODUCTION: A GLOBAL VIEW

*From send/receive to cooperation abstractions.* The operations send() and receive() constitute the machine language of underlying networks. So, in order to solve a distributed computing problem it is usual to first define an appropriate communication abstraction that makes easier the design of higher level algorithms. Fifo and Causal message-ordering [4, 11] are examples of such communication abstractions that make easier the construction of distributed objects such as, for example, a causal memory on top of an asynchronous message-passing system [1]. A well-know high level and very powerful communication abstraction is total order broadcast, which ensures that the message delivery order is the same at all processes.

*On the read/write side.* Read/write (RW) registers (i.e., the cells of a Turing machine) are at the center of distributed algorithms when the processes communicate through a shared memory. So a fundamental problem is the construction of atomic RW registers on top of a crash-prone asynchronous message-passing system. This problem has been solved by Attiya, Bar-Noy and Dolev who presented in [3] a send/receive-based algorithm (ABD) for such a construction, and proved that, from an operational point of view, such constructions are possible if and only if at most $t < n/2$ processes may crash. The ABD construction is based on the explicit use of sequence numbers, quorums and a send/receive pattern (used once to write and twice to read). The quorums are used to realize synchronization barriers.

Based on the same principles as ABD, algorithms building RW registers have been designed. Some strive to reduce the size of control information carried by messages (e.g. [2, 8–10]), others focus on fast read and/or fast write operations in good circumstances, others consider appropriate communication abstractions (e.g., [8]) and others consider specific models [7]. All these algorithms allow existing RW-based algorithms to be used on top of crash-prone asynchronous message-passing systems.

The use of RW registers on top of a message-passing system to allow processes to use "for free" existing shared memory-based distributed algorithms has a cost, which can be higher than the one obtained with an algorithm directly designed on top of the basic send/receive operations. This means that, for some problems, algorithms based on appropriate communication abstractions can be more efficient than the stacking of RW-based algorithms on top of simulated RW registers. In such a context the proposed MBroadcast has the following properties:

- It has the same computability power as RW registers.
- It constitutes the first characterization of RW registers in terms of (binary) message patterns.
- It allows to build higher level coordination abstractions without requiring as prerequisite the construction of an intermediary abstraction level made up of RW registers.

Publication rights licensed to ACM. ACM acknowledges that this contribution was authored or co-authored by an employee, contractor or affiliate of a national government. As such, the Government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for Government purposes only.
*PODC '23, June 19–23, 2023, Orlando, FL, USA*
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0121-4/23/06…$15.00
https://doi.org/10.1145/3583668.3594569

- Its cost is $O(n)$ implementation messages when mutual broadcast delivery concerns only correct processes.
- It makes quorums useless at the upper layer.
- When looking at a message exchange between two processes $p$ and $p'$, it shows that the fact that $p'$ does not ignore the other process $p$ (because it received a message from $p$ before receiving its own message) is a powerful control information.

The full version of this article is available online [5]. It includes more discussions about the interpretation of MBroadcast, as well as many other algorithms that illustrate the usage of MBroadcast, and the proofs of the algorithms presented thereafter.

## 2 DISTRIBUTED COMPUTING MODEL

*Process model.* The computing model is composed of a set of $n$ sequential processes denoted $p_1, ..., p_n$. Sometimes, when considering two processes, they are denoted $p$ and $p'$.

Each process is asynchronous which means that it proceeds at its own speed, which can be arbitrary and remains always unknown to the other processes. A process may halt prematurely (crash failure), but executes correctly its local algorithm until it possibly crashes. The model parameter $t$ denotes the maximal number of processes that may crash in a run. A process that crashes in a run is said to be *faulty*. Otherwise, it is *correct* or *non-faulty*.

*Communication model.* Each pair of processes communicate by sending and receiving messages through two uni-directional channels, one in each direction. Hence, the communication network is a complete network: any process $p_i$ can directly send a message to any process $p_j$ (including itself). A process $p_i$ invokes the operation "send TYPE($m$) to $p_j$" to send the message $m$ (whose type is TYPE) to $p_j$. The operation "receive TYPE($m$) from $p_j$" allows $p_i$ to receive from $p_j$ a message $m$ whose type is TYPE.

Each channel is reliable (no loss, no corruption, no creation of messages), not necessarily first-in/first-out, and asynchronous (while the transit time of each message is finite, there is no upper bound on message transit times). Let us notice that, due to process and message asynchrony, no process can know if another process crashed or is only very slow.

It is assumed that, in addition to basic send/receive operations, the network is enriched with FIFO and causal message deliveries, i.e. it provides the processes with the operations fifo_broadcast(), causal_broadcast(), causal_send(), and causal_delivery() [4, 6, 11]. Two messages $m$ and $m'$ are causally related if (i) they have been sent by the same process and $m$ was sent before $m'$, or (ii) a process received $m$ before sending $m'$, or (iii) there is chain of messages $m$, $m_1$, ..., $m_x$, $m'$ such that each pair of consecutive messages is causally related by (i) or (ii). It is important to note that the addition of this assumption does not change the computability power of the communication model, since causal message delivery can always be implemented on top of send/receive channels. It is also important to note that, while the implementation of these operations requires messages to carry additional control information, it does not require the use of additional implementation messages.

## 3 MUTUAL BROADCAST: DEFINITION

Mutual-broadcast (MBroadcast) is a communication abstraction that allows a process to broadcast a message that will be delivered at least by all the correct processes. This abstraction provides the processes with two operations denoted mbroadcast() and mdeliver(). When a process invokes mbroadcast($m$) we say "it mbroadcasts the message $m$". The invocation of mdeliver() returns a message $m$ and we say that a process "mdelivers $m$" or that "$m$ is mdelivered". To simplify the presentation (and without loss of generality), it it assumed that all the messages that are mbroadcast are different. The following properties define MBroadcast.

**Validity.** If a process $p_i$ mdelivers a message $m$ from a process $p_j$, then $p_j$ previously invoked mbroadcast($m$).

**No-duplication.** A process mdelivers a message $m$ at most once.

**Mutual ordering.** For any pair of processes $p$ and $p'$, if $p$ mbroadcasts a message $m$ and $p'$ mbroadcasts a message $m'$, it is not possible that $p$ mdelivers $m$ before $m'$ and $p'$ mdelivers $m'$ before $m$.

**Local termination.** If a correct process invokes mbroadcast($m$), it returns from its invocation.

**Global CS-termination.** If a correct process invokes mbroadcast($m$), all correct processes mdeliver $m$. ("CS" is used to stress the fact that the sender is required to be correct.)

*Additional properties.* As it is the case with other broadcast abstractions, MBroadcast can be enriched with other properties, defined thereafter, that do not change its computing power but add more usage comfort in some algorithms. We denote by fifo-MBroadcast, causal-MBroadcast and reliable-MBroadcast the MBroadcast abstraction respectively enriched with the following properties:

**FIFO ordering.** If a process mbroadcasts a message $m$ and then mbroadcasts a message $m'$, no process mdelivers $m'$ before $m$.

**Causal ordering.** If a process mdelivers a message $m$ and then mbroadcasts a message $m'$, no process mdelivers $m'$ before $m$.

**Global CF-termination.** If a process mdelivers a message $m$, all correct processes mdeliver $m$. ("CF" is used to stress the fact that the receiver of $m$ can be correct or faulty.)

*Remark on reliable broadcast.* It is easy to see that the classical reliable broadcast communication abstraction is MBroadcast with the Global CF-termination property and without the Mutual ordering property. Similarly, the causal broadcast abstraction (denoted causal_broadcast in the following) is MBroadcast with the Causal ordering and Global CF-termination properties and without the Mutual ordering property.

*A real-time property.* When combined with the fact that a message cannot be received before it has been sent, MBroadcast ensures that if a process $p$ mdelivers a message $m$ it has mbroadcast before a process $p'$ mbroadcast a message $m'$, $p'$ cannot mdeliver $m'$ before $m$. This pattern is described in Figure 1. In other words, sending a "blank" synchronization message and waiting for its delivery is sufficient to "harvest" all the messages that were already mdelivered by their senders. To benefit from this property, we consider the synchronized broadcast operation based on a synchronization barrier as defined in Algorithm 1.

```
operation synchro_mbroadcast(m) is      % code for p_i
(1)  mbroadcast(m);
(2)  wait (m has been mdelivered from p_i)
end operation.
```

**Algorithm 1: Synchronized MBroadcast**

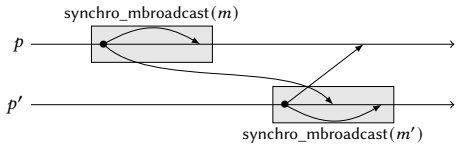



**Figure 1: A message delivery real-time property**

The operation synchro_mbroadcast() inherits the properties defining MBroadcast, as well as the following property.

Property 1. *If a process p returns from* synchro_mbroadcast$(m)$ *before a process $p'$ invokes* synchro_mbroadcast$(m')$*, then process $p'$ cannot return from* synchro_mbroadcast$(m')$ *before it has mdelivered m.*

## 4 MBROADCAST: AN IMPLEMENTATION

*Description of the algorithm.* Algorithm 2 implements MBroadcast in an asynchronous $n$-processes system where up to $t < n/2$ processes may crash. When $p_i$ invokes mbroadcast$(m)$ it issues a causal broadcast of the implementation message INIT$(m)$, and waits until it has received corresponding acknowledgment from $(n - t - 1)$ processes. When this occurs it locally mdelivers $m$. Finally, when a process $p_i$ causally delivers a message INIT$(m)$ from another process $p_j$, it causally acknowledges its receipt and locally mdelivers $m$.

```
operation mbroadcast(m) is                          % code for p_i
(1)  causal_broadcast INIT(m);
(2)  wait (ACK(m) received from at least n − t − 1 processes);
(3)  mdelivery of m from p_i
end operation.

when INIT(m) is causally-delivered from p_j do
(4)  if (i ≠ j) then
(5)      causal_send ACK(m) to p_j;
(6)      mdelivery of m from p_j
(7)  end if.
```

**Algorithm 2: MBroadcast on top of $\mathcal{CAMP}_{n,t}[t < n/2]$**

Theorem 1. *Algorithm 2 builds the* MBroadcast *communication abstraction in the distributed system model $\mathcal{CAMP}_{n,t}[t < n/2]$.*

## 5 IMPLEMENTING ATOMIC RW REGISTERS WITH MBROADCAST

*Local variables.* Each process $p_i$ manages three local variables.

- $val_i$ contains the current value of the register as known by $p_i$.
- $t_i$ is the date of the last write issued by $p_i$.
- $clock_i$ is a pair (timestamp) $\langle date, writer\rangle$ defining the identity of the value in $val_i$ as a timestamp: $writer$ is the identity of the writer of $val_i$ and $date$ the associated Lamport's logical date. Let us remind that all the logical dates are totally ordered according to the usual lexicographical order.

*Description of the algorithm.* Algorithm 3 implements an MWMR atomic register on top of MB-broadcast, i.e., in the model $\mathcal{CAMP}_{n,t}$[MB-broadcast].

```
operation write(v) is                          % code for p_i
(1)  synchro_mbroadcast SYNCH();
(2)  let t_i such that ⟨t_i, i⟩ > clock_i;
(3)  synchro_mbroadcast WRITE(v, ⟨t_i, i⟩)
end operation.

operation read() is
(4)  synchro_mbroadcast SYNCH();
(5)  v ← val_i;
(6)  synchro_mbroadcast WRITE(v, clock_i);
(7)  return(v)
end operation.

when WRITE(v, c) is mb-delivered from p_j do
(8)  if c > clock_i then val_i ← v; clock_i ← c end if.
```

**Algorithm 3: Atomic RW register on top of the model $\mathcal{CAMP}_{n,t}$[MB-broadcast]**

When $p_i$ invokes write(), it first mb-broadcasts a pure control message SYNCH() to resynchronize its local state with respect to possible write operations that modify the last value of the register (line 1). Then, it computes the timestamp associated with the value it wants to write (line 2), and propagates its write operation with a synchronized broadcast (line 3).

The read operation is similar. Lines 4-5 provide $p_i$ with the value it has to return, while line 6 implements the "reads have to write" strategy needed to ensure atomicity of the read operation [3]. Finally, when $p_i$ mb-delivers a WRITE() message, it updates its local context if the new timestamp is higher than the one it currently has in its local variables.

Theorem 2. *Algorithm 3 implements an* atomic RW register *on top of $\mathcal{CAMP}_{n,t}$*[MB-broadcast].

Conversely, MBroadcast can be implemented using atomic RW registers, which shows that the two abstractions have the same computational power.

*Important remark.* It is important to state that neither the construction of an atomic read/write register on top of MBroadcast, nor the construction of MBroadcast on top of atomic read/write registers, needs the assumption $t < n/2$. In this sense, MBroadcast is a characterization of read/write registers in crash-prone asynchronous message-passing systems.

## ACKNOWLEDGMENTS

This work was partially supported by the French "Étoile Montante en Pays De La Loire" regional project BROCCOLI devoted to the computability aspects of broadcast abstractions, and the French ANR project ByBLoS (ANR-20-CE25-0002-01) devoted the modular design of building blocks for large-scale trustless multi-users applications.

# REFERENCES


[1] Mustaque Ahamad, Gil Neiger, James E Burns, Prince Kohli, and Phillip W Hutto. 1995. Causal memory: Definitions, implementation, and programming. *Distributed Computing* 9, 1 (1995), 37–49.

[2] Hagit Attiya. 2000. Efficient and robust sharing of memory in message-passing systems. *Journal of Algorithms* 34, 1 (2000), 109–127.

[3] Hagit Attiya, Amotz Bar-Noy, and Danny Dolev. 1995. Sharing memory robustly in message-passing systems. *Journal of the ACM (JACM)* 42, 1 (1995), 124–142.

[4] Kenneth P Birman and Thomas A Joseph. 1987. Reliable communication in the presence of failures. *ACM Transactions on Computer Systems (TOCS)* 5, 1 (1987), 47–76.

[5] Mathilde Déprés, Achour Mostefaoui, Matthieu Perrin, and Michel Raynal. 2023. Send/Receive Patterns versus Read/Write Patterns: the MB-Broadcast Abstraction (Extended Version). (May 2023). https://hal.science/hal-04087447 working paper or preprint.

[6] Vassos Hadzilacos and Sam Toueg. 1994. *A modular approach to fault-tolerant broadcasts and related problems.* Technical Report. Cornell University.

[7] Maurice Herlihy, Sergio Rajsbaum, Michel Raynal, and Julien Stainer. 2017. From wait-free to arbitrary concurrent solo executions in colorless distributed computing. *Theoretical Computer Science* 683 (2017), 1–21.

[8] Damien Imbs, Achour Mostéfaoui, Matthieu Perrin, and Michel Raynal. 2021. Set-constrained delivery broadcast: A communication abstraction for read/write implementable distributed objects. *Theoretical Computer Science* 886 (2021), 49–68.

[9] Achour Mostéfaoui and Michel Raynal. 2016. Two-bit messages are sufficient to implement atomic read/write registers in crash-prone systems. In *Proceedings of the 2016 ACM Symposium on Principles of Distributed Computing*. ACM, 381–389.

[10] Achour Mostéfaoui, Michel Raynal, and Matthieu Roy. 2019. Time-efficient read/write register in crash-prone asynchronous message-passing systems. *Computing* 101 (2019), 3–17.

[11] Michel Raynal. 2018. *Fault-tolerant message-passing distributed systems: an algorithmic approach.* Springer.